ROS 2

Abstract—The Robot Operating System 2 (ROS 2) is an open-source middleware framework for building distributed robot software. This report explains the ROS 2 computation graph and its core entities: nodes, topics, services, actions and parameters. It compares topics with services and states when each should be used. It then covers custom interfaces, the colcon build tool, workspace structure and Quality of Service (QoS) settings. Finally, it presents two solutions developed in Python (rclpy): a wheelspeed publisher/subscriber using a custom TaskOne message, and a three-node palindrome checker using a custom RT2String message, each with its ROS graph.

Concepts:

—ROS 2 middlewarre

DDS

publish–subscribe

custom interfaces

colcon

Quality of Service.

.

I. Introduction

ROS 2 lets a robot's software be split into small, independent programs that exchange data over a common middleware [1], [2]. A driverless vehicle, for example, runs perception, state estimation, planning and control as separate processes that must share data reliably and in real time.

ROS 1 relied on a central master node, which was a single point of failure and offered no real-time or security guarantees [3]. ROS 2 replaced the master with the Data Distribution Service (DDS) standard [4], giving decentralised discovery, configurable QoS, security and support for Linux, Windows and macOS. The current release is ROS 2 Lyrical Luth (May 2026), the twelfth ROS 2 distribution [5].

Section II describes the computation graph. Section III covers the communication mechanisms. Section IV covers custom interfaces and the build system, and Section V covers QoS. Sections VI and VII present the two task solutions, and Section VIII concludes.

II. The ROS 2 Computation Graph and Nodes

A. Computation Graph

The computation graph is the network of ROS 2 elements processing data together at runtime [1]. Its vertices are nodes and its edges are the topics, services and actions that connect them. There is no master: each node discovers the others automatically through DDS, provided they share the same ROS_DOMAIN_ID. The graph can be inspected with ros2 node list and ros2 topic list, and visualised with rqt_graph.

B. Nodes

A node is a single-purpose executable unit, such as a camera driver or a speed controller. Each node can publish and subscribe to topics, offer or call services, run action servers or clients, and hold parameters. In Python a node is a subclass of rclpy.node.Node; it is started with rclpy.init(), kept alive with rclpy.spin() and shut down with rclpy.shutdown().

III. Communication Mechanisms

A. Topics

A topic is a named bus over which nodes exchange messages in a publish–subscribe pattern [1]. Communication is asynchronous and many-to-many: any number of publishers and subscribers may share a topic, and none of them knows about the others. Each topic carries one message type, e.g. sensor_msgs/msg/Image. Useful tools are ros2 topic echo, pub and hz.

B. Services

A service is a synchronous request–response call between a client and a single server. The client sends a request and receives exactly one response. Service types are defined in .srv files with the request and response separated by . Tools: ros2 service list, ros2 service call.

c.Action*s*

An action is used for long-running, goal-oriented tasks. A client sends a goal, receives periodic feedback while the server works, and finally gets a result; the goal can also be cancelled. Internally an action is built from three services (send goal, cancel, get result) and two topics (feedback, status). Action types are defined in .action files with goal, result and feedback sections.

d.Parameters

Parameters are per-node configuration values (integers, floats, booleans, strings and arrays). They are declared in code with declare_parameter(), can be set at launch or from a YAML file, and changed at runtime with ros2 param set. A typical use is a controller gain that is tuned without recompiling.

*E.*Topic vs Service: When to Use Each

The main difference between the two is the pattern of communication. A topic follows the publish–subscribe model, while a service follows the request–response model. Topics are asynchronous and carry a continuous stream of data, whereas services are synchronous and are called only when needed. A topic can have many publishers and many subscribers, but a service has a single server that may be called by many clients. Also, a publisher never receives a reply, while every service request gets exactly one response. Typical topic data includes sensor readings, odometry and velocity commands; typical services include resetting odometry, switching a mode or querying a value.

A topic should be used when data is produced continuously and consumers only need the latest value, such as LiDAR scans or wheel speeds. A service should be used for occasional, short operations where the caller needs a confirmation or result before continuing. Anything that takes long or needs progress updates, such as navigating to a waypoint, should be an action instead.

IV. Custom Interfaces, colcon and Workspace Structure

A. *Custom Interfaces*

a custom interface is a data structure that YOU create to define what information your nodes exchange.

For them to communicate, they need to agree on the format of the data being sent. That format is called an interface.

ROS 2 already provides built-in interfaces, such as String, Int32, and Twist. But sometimes your application needs a specific combination of data.

For example, imagine you want to send this information together:

- Vehicle speed
- Steering angle
- Brake status

A custom message could define all three fields:

float32 speed
float32 steering_angle
bool brake_active

Types of custom interfaces

Type	Used for	Example
.msg	Sending data	Sending vehicle speed
.srv	Request and response	Asking a sensor to reset
.action	Long-running tasks with feedback	Navigating to a target location

Basically A custom interface in ROS 2 is a user-defined data format that allows nodes to exchange application-specific information in a structured and consistent way.

B. *Workspace Structure*

A workspace is a directory holding ROS 2 packages. Only src/ is written by the developer; colcon creates the other three folders:

```
ros2_ws/
|-- src/                    # source packages
|       |-- task_interfaces/ # ament_cmake: msg/
|       `-- task_nodes/      # ament_python: nodes
|-- build/    # intermediate build files
|-- install/                # built packages + setup scripts
`-- log/                    # build logs
```

C. *colcon*

colcon is the ROS 2 build tool . It builds every package in src/ in dependency order:

```
cd ~/ros2_ws
colcon build                          # build all packages
colcon build --packages-select task_nodes source install/setup.bash
# overlay the workspace
```

What happens when you run colcon build?

Three important folders are typically created or updated:

Folder	Purpose
build/	Intermediate build files
install/	Built and installed packages
log/	Build logs and error details

. Important colcon commands

A. Build the entire workspace

cd ~/driverless_ws

colcon build

Builds the packages found in your workspace's src/ directory.

B. Build only one package

colcon build --packages-select task2_nodes

Useful when you only want to rebuild task2_nodes.

C. Build a package and its dependencies

colcon build --packages-up-to task2_nodes

Builds the selected package and the workspace packages it depends on.

D. Build with symbolic links for Python packages

colcon build --symlink-install

The workspace is an overlay on top of the underlay (the installed ROS 2 distribution), so the underlay is sourced first and then install/setup.bash. New packages are created with ros2 pkg create build-type ament_python <name> or buildtype ament_cmake..

Basically colcon is the build tool that builds your ROS 2 packages inside a workspace.

V. Quality of Service (QoS) Settings

I

n ROS 2, QoS (Quality of Service) settings control how messages are delivered between nodes.

Think of QoS as rules that decide whether messages should be stored, how reliably they should be delivered, and how long they should be kept.

QoS policies let each publisher and subscriber choose how reliably and how long data is delivered . This lets ROS 2 work over lossy Wi-Fi as well as wired links. The main policies are as follows.

1. Why do we need QoS?

Imagine a LiDAR sensor on a Formula Driverless car publishing data continuously.

- If one message is lost, the next message may arrive a moment later.
- Keeping every old message could cause the controller to process outdated information.
- For important commands, you may prefer reliable delivery rather than risk losing a message.

QoS lets you choose the appropriate communication behavior for each situation.

History can be set to keep the last N samples or keep all samples, and decides how many messages are queued. Depth is the value of N, i.e. the queue size when keep-last is used. Reliability can be reliable, where lost messages are re-sent, or best effort, where they are simply dropped. Durability can be volatile or transient local; with transient local, a subscriber that joins late still receives the messages published before it. Deadline sets the maximum expected gap between two messages, and Lifespan sets how old a message can get before it is discarded. Finally, Liveliness decides how a publisher shows that it is still alive, either automatically or manually.

Publisher and subscriber profiles must be compatible. For example, a reliable subscriber cannot receive from a best-effort publisher, and the mismatch silently produces no data; ros2 topic info -v shows each endpoint's profile.

ROS 2 provides preset profiles. The default (Keep last 10, Reliable, Volatile) suits commands and the tasks in this report. The sensor-data profile (best effort, small depth) suits high-rate sensors where the newest sample matters more than every sample. Transient local durability suits data published once, such as a static map. In rclpy the depth is the integer passed to create_publisher(), or a full QoSPro le object can be given.

VI. Solution 1: Wheel Speed with a Custom TaskOne Message

Step 1.1: Open a terminal and load ROS2 Every new terminal must "load" ROS2 first, like logging in. source /opt/ros/humble/setup.bash

Step 1.2: Make a workspace mkdir -p ~/ros2_ws/src cd ~/ros2_ws/src "src" is where all your packages go. We will create all four packages inside it.

Step 1.3: Check that ROS2 works ros2 --help If you see a list of commands, you are ready

2. Q1: TaskOne publisher and subscriber

Step 2.1: Create the message package cd ~/ros2_ws/src ros2 pkg create --build-type ament_cmake taskone_interfaces mkdir taskone_interfaces/msg

Step 2.2: Write the message file Create the file taskone_interfaces/msg/TaskOne.msg with exactly these two lines: float32 ang_vel float32 radius

Step 2.3: Edit CMakeLists.txt Open taskone_interfaces/CMakeLists.txt, delete everything, and paste: cmake_minimum_required(VERSION 3.8) project(taskone_interfaces) find_package(ament_cmake REQUIRED) find_package(rosidl_default_generators REQUIRED) rosidl_generate_interfaces(${PROJECT_NAME} "msg/TaskOne.msg") ament_export_dependencies(rosidl_default_runtime) ament_package()

(Think of CMakeLists.txt as the **instruction sheet for the builder**. When you run colcon build, ROS2 reads this file to learn what to build and how.

Why it's needed for the message package

Your TaskOne.msg file is just a text file with two lines:

float32 ang_vel

float32 radius

Python can't use a text file like that directly. ROS2 has to turn it into real code (Python and C++ classes) so you can write from taskone_interfaces.msg import TaskOne and use msg.ang_vel. That conversion is called

code generation, and it only happens if you ask for it in CMakeLists.txt.)

Step 2.4: Edit package.xml

Open taskone_interfaces/package.xml and add these three lines just above the line that says export: rosidl_default_generators rosidl_default_runtime rosidl_interface_packages

Step 2.5: Create the node package

cd ~/ros2_ws/src

ros2 pkg create --build-type ament_python taskone_pkg \

--dependencies rclpy taskone_interfaces

Your Python files go in taskone_pkg/taskone_pkg/ (the inner folder that contains __init__.py)

The command, piece by piece

bash

cd ~/ros2_ws/src

ros2 pkg create --build-type ament_python taskone_pkg \

--dependencies rclpy taskone_interfaces

cd ~/ros2_ws/src	**Go into the src folder, where all packages must live**
ros2 pkg create	**"ROS2, please make a new package folder with the starter files"**
--build-type ament_python	**"This package holds Python code" (the message package used ament_cmake)**
taskone_pkg	**The name of the new package**
--dependencies rclpy taskone_interfaces	**"My code needs these two things"**

Step 2.6:

Write the publisher Create taskone_pkg/taskone_pkg/taskone_pub.py:

```
import rclpy
from taskone_interfaces.msg import TaskOne
def main(args=None):
rclpy.init(args=args)
node = rclpy.create_node('taskone_publisher')
pub = node.create_publisher(TaskOne, '/taskone', 10
) try
: while rclpy.ok():
try:
w = float(input('Enter angular velocity (rad/s): '))
r = float(input('Enter radius (m): '))
except ValueError:
print('Please type numbers only!')
continue msg = TaskOne()
msg.ang_vel = w
msg.radius = r
pub.publish(msg)
node.get_logger().info(f'Published: ang_vel={w}, radius={r}')
except (KeyboardInterrupt, EOFError):
pass
node.destroy_node()
rclpy.try_shutdown()
if __name__ == '__main__':
main()
```

• input() waits for you to type in the terminal.

• float() turns your typed text into a number; if you type letters it asks again instead of crashing.

• pub.publish(msg) puts the note on the /taskone notice board.

Step 2.7: Write the subscriber Create taskone_pkg/taskone_pkg/taskone_sub.py:

```
import rclpy
from rclpy.node import Node
from taskone_interfaces.msg import TaskOne


class TaskOneSubscriber(Node):
    def __init__(self):
        super().__init__('taskone_subscriber')
        self.create_subscription(TaskOne, '/taskone', self.callback, 10)

    def callback(self, msg):
        speed = msg.ang_vel * msg.radius      # v = w * r
        self.get_logger().info(
            f'Received ang_vel={msg.ang_vel}, radius={msg.radius} '
            f'-> Longitudinal speed = {speed:.3f} m/s')


def main(args=None):
    rclpy.init(args=args)
    node = TaskOneSubscriber()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    node.destroy_node()
    rclpy.try_shutdown()


if __name__ == '__main__':
    main()
```

The callback function runs automatically every time a new note arrives on /taskone. rclpy.spin(node) keeps the node alive and listening.

Step 2.8: Register the programs in setup.py

Open taskone_pkg/setup.py, find entry_points and change it to:

entry_points=

{ 'console_scripts': [

'taskone_pub = taskone_pkg.taskone_pub:main',

'taskone_sub = taskone_pkg.taskone_sub:main',], },

This gives each program a short name you can use with ros2 run

Step 2.9: Build

cd ~/ros2_ws

colcon build --packages-select taskone_interfaces taskone_pkg

source install/setup.bash

Check that the message was created:

ros2 interface show taskone_interfaces/msg/TaskOne

Expected output:

float32 ang_vel

float32 radius

Step 2.10:

Run Q1

Open two terminals. In each one, first run:

source /opt/ros/humble/setup.bash

source ~/ros2_ws/install/setup.bash

Why you need 2 terminals

Each node is its own program, and each one keeps running until you stop it. A terminal can only run one program at a time, so you open one terminal per node.

Step A: Prepare both terminals

Open two terminals. In **each** one, type:

bash

source /opt/ros/humble/setup.bash

source ~/ros2_ws/install/setup.bash

tep B: Terminal 1 (subscriber)

bash

ros2 run taskone_pkg taskone_sub

Step C: Terminal 2 (publisher)

bash

ros2 run taskone_pkg taskone_pub

What you should see

Terminal 2 (publisher):

[INFO] [taskone_publisher]: Published: ang_vel=10.0, radius=0.3

Enter angular velocity (rad/s):

Terminal 1 (subscriber):

[INFO] [taskone_subscriber]: Received ang_vel=10.0, radius=0.3000000119

-> Longitudinal speed = 3.000 m/s

Step 2.11: rqt_graph screenshot for Q1

With both nodes still running, open a third terminal:

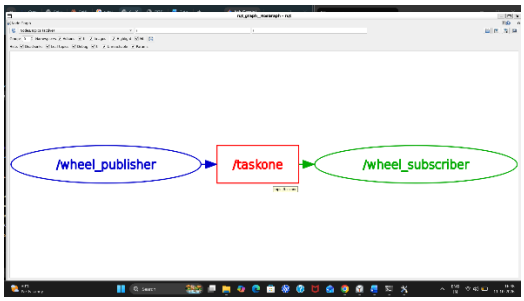
source /opt/ros/humble/setup.bash rqt_graph

• Click the blue refresh button (circular arrow) in the top-left.

• In the dropdown, choose Nodes/Topics (all).

• You should see: taskone_publisher -> /taskone -> taskone_subscriber.

• Press the Print Screen key (or use the Screenshot app) and save the image as q1_rqt_graph.png.



3. Q2: Three-node palindrome checker

VII. Solution 2: Three-Node Palindrome Checker

A. *Problem*

Node 1 continuously publishes user-entered strings (e.g. “AABAA”, then “BACAB”) using a custom message RT2String. Node 2 checks each string for a palindrome and sends 1 (true) or 0 (false) to Node 3. Node 3 replies “Yes” or “No” to Node 2, which prints the result.

B. *Design*

Three topics connect the nodes. Node 1 sends the user’s string to Node 2 on /input_string using the custom RT2String message. Node 2 sends 1 or 0 to Node 3 on /palindrome_result using the standard std_msgs/Int32 message. Node 3 sends “Yes” or “No” back to Node 2 on /palindrome_reply, again as an RT2String. The file task_interfaces/msg/RT2String.msg holds just one field:

string data

Node 2 compares the string with its reverse (s == s[::-1]); the check is case-sensitive, so “Abba” returns 0. Node 2 remembers the last string so it can print it next to Node 3’s reply. Each node uses the same main() pattern as Solution 1; only the classes are shown below, and package.xml adds <depend>std_msgs</depend>.

C. *Node 1: String Publisher*

from task_interfaces.msg import RT2String

class StringPublisher(Node):

```
    def __init__(self): super().__init__('string_publisher')
        self.pub = self.create_publisher(
            RT2String, '/input_string', 10)
    def run(self): while
        rclpy.ok(): msg =
        RT2String()
        msg.data =
        input('Enter a
        string: ').strip()
        self.pub.publish(ms
        g)
```

D. Node 2: Palindrome Checker

```
from std_msgs.msg import Int32 from
task_interfaces.msg import RT2String


class PalindromeChecker(Node):
    def __init__(self): super().__init__('palindrome_checker')
        self.last = '' self.create_subscription(
            RT2String, '/input_string', self.on_string, 10) self.result_pub =
        self.create_publisher(
            Int32, '/palindrome_result', 10) self.create_subscription(
            RT2String, '/palindrome_reply', self.on_reply, 10)
    def on_string(self, msg): self.last =
        msg.data result = Int32()
        result.data = 1 if msg.data == msg.data[::-1] else 0
        self.result_pub.publish(result)

    def on_reply(self, msg):
        self.get_logger().info(
            f "{self.last}" is a palindrome: {msg.data}')
```

E. Node 3: Result Responder

```
from std_msgs.msg import Int32 from
task_interfaces.msg import RT2String


class ResultResponder(Node):
    def __init__(self): super().__init__('result_responder')
        self.create_subscription(
            Int32, '/palindrome_result', self.on_result, 10) self.reply_pub =
        self.create_publisher(
            RT2String, '/palindrome_reply', 10)
    def on_result(self, msg): reply = RT2String() reply.data =
        'Yes' if msg.data == 1 else 'No'
        self.reply_pub.publish(reply)
```

F. Run and Output

```
ros2 run task_nodes string_publisher  # terminal 1 ros2 run task_nodes
palindrome_checker       # terminal 2 ros2 run task_nodes
result_responder         # terminal 3
```

Entering “AABAA” prints "AABAA" is a palindrome: Yes; “BACAB” also prints Yes, and “ABCD” prints No.

G. Discussion

The exchange between Nodes 2 and 3 is really a question and an answer, which Section III-E maps to a service. Topics are used here to follow the task specification; a /check_palindrome service would remove the need for Node 2 to remember the last string.

H. ROS Graph

The graph (Fig. 2) shows a loop: Node 2 both publishes to and subscribes from Node 3.

VIII. Conclusion

ROS 2 structures robot software as a decentralised graph of nodes that communicate through topics, services and actions,

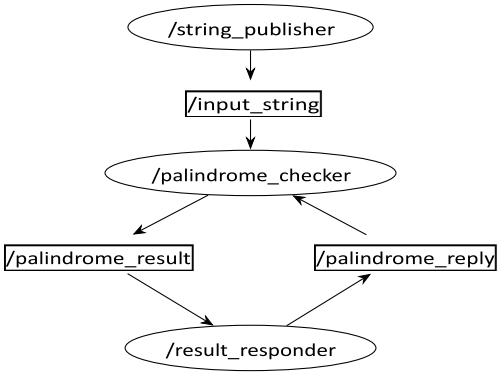


Fig. 2. ROS graph for Solution 2 (ovals: nodes; rectangles: topics).

configured by parameters and tuned by QoS. Custom interfaces and the colcon workspace make it straightforward to add application-specific data types. The two solutions apply these ideas: Solution 1 streams wheel measurements over one topic, and Solution 2 chains three nodes through a feedback loop. Both confirm that choosing the right communication pattern is the main design decision in a ROS 2 system.

